

정렬 비교 보고서 — 병합 · 퀵 · 힙

2026-2 고급알고리즘(SIT2001-01) 1차 과제 · 2026193020 김성혁

- 저장소: https://github.com/SungHyuk108/Advanced_algorithm_SungHyukKim_Comparing_Sorting
- 구현: [src/mergeSort.c](#) · [src/quickSort.c](#) · [src/heapSort.c](#)
- 실행: `make run` 측정 표 · `make test` 유닛 테스트 · `make charts` 그래프 생성
- 환경: 컨테이너 안에서 `gcc -std=c17 -Wall -Wextra -O2` (컴파일 경고 0건)

차례

1. 개요 — 무엇을 왜 비교했나
2. 본론 - 2.1 코드 전반적인 구조 분석 - 2.2 정렬별 동작 과정 - 2.3 알고리즘 실험 - 2.4 공간 복잡도
3. 소감 및 결론

1. 개요

이번 정렬 알고리즘 과제에서 비교하고 싶었던 것은 세 가지였다. **합치는 데 비용이 드는 병합 정렬**, **나누는 데 비용이 드는 퀵 정렬**, 그리고 나누지도 합치지도 않고 "힙"이라는 독특한 구조 아래에서 작동하는 **힙 정렬**이다.

더 정확히 말하면, 강의 교본의 "**나누기와 결합은 시소**" 라는 표현을 기반으로 이 정렬들을 선택했다고 봐도 무방하다. 한쪽에서 비용을 치르면 다른 쪽은 비용이 들지 않는다는 것이 그 틀인데, 병합과 퀵이 그 시소의 양 끝이다. 배우지 않은 정렬로 힙을 고른 이유도 여기에 있다. 힙은 시소의 어느 쪽도 아니다 — **시소 자체를 벗어난 세 번째 경우**다.

	정렬	수업에서	나누기	결합	치르는 대가
1	병합 정렬	배움	비용 없음 (위치로 반 자름)	여기서 비용 발생	temp 배열 $O(n)$
2	퀵 정렬	배움	여기서 비용 발생 (값으로 가름)	비용 없음	최악 $O(n^2)$ · 불안정
3	힙 정렬	안 배움	—	—	무작위 입력에서 가장 느림

세 정렬을 같은 조건에서 돌리고 **실행 시간 · 비교 횟수 · 이동 횟수 · 재귀 깊이** 네 가지를 잴다.

2. 본론

2.1 코드 전반적인 구조 분석

코드의 전반적인 구조는, 교수님께서 바이트 코딩을 강조하셨듯이 교수님께서 올려놓으신 샘플 보고서와 코드 형식을 참고하여 진행하였다.

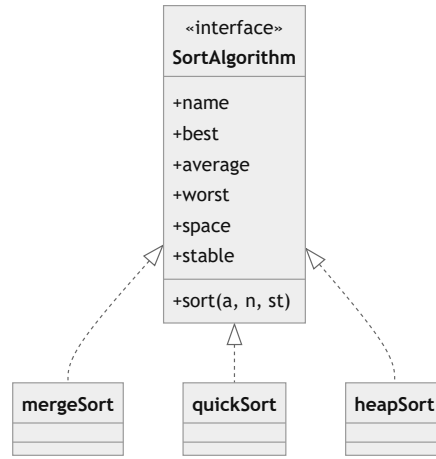
기본적으로 세 정렬을 제대로 분석하고 측정하려면 **각 정렬마다 측정 코드를 따로 두는 것이 아니라 하나의 공통된 인터페이스가 필요하겠**다는 판단에서 출발했다. 정렬마다 재는 방식이 조금씩 달라지면 나온 숫자를 나란히 놓을 근거가 확보할 수 없기 때문이다. 그래서 `sort.h`를 먼저 만들고, 여러 정렬 알고리즘을 그 약속에 맞춰 구현한 뒤, 그것을 `bench.c`에 넣어 측정하고 결과를 `main.c`가 찍게 하였다.

공통 인터페이스

함수 포인터를 멤버로 가진 구조체를 하나 정의해 둬으로써 해당 역할을 수행할 수 있게 하였다.

```
/* src/sort.h */
typedef struct SortAlgorithm {
    const char *name;
    const char *best;    /* 최선 시간복잡도 */
    const char *average; /* 평균 */
    const char *worst;   /* 최악 */
    const char *space;   /* 추가 공간 */
    int stable;          /* 안정 정렬이라고 주장하는 값. 테스트가 실측과 맞춰 본다 */
    void (*sort)(Record *a, size_t n, SortStats *st);
} SortAlgorithm;

extern const SortAlgorithm SORT_ALGORITHMS[]; /* 구현 목록 */
```

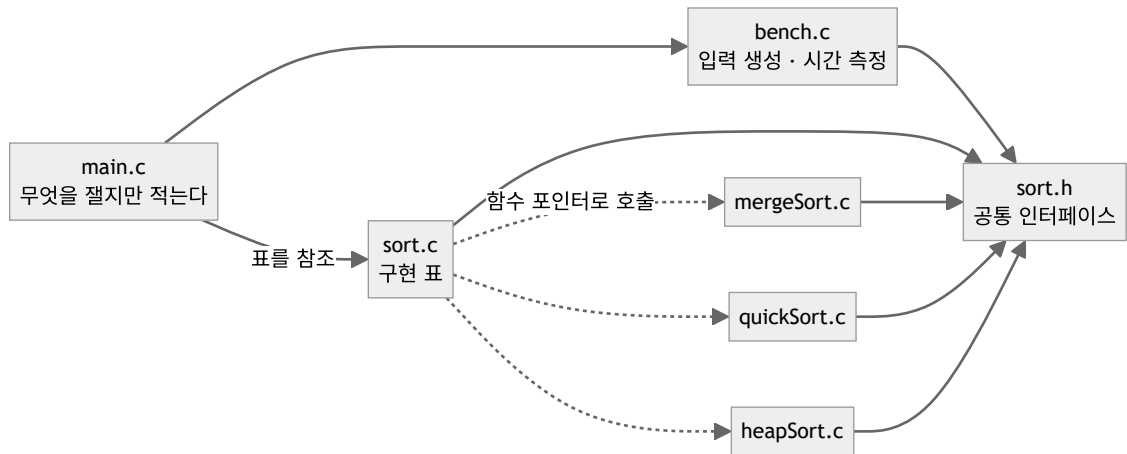


C에는 상속이 없으므로 이 표가 곧 구현 목록이다. 정렬을 하나 추가한다는 것은 .c 파일 하나를 쓰고 아래 표에 한 줄을 더한다는 뜻이 된다.

```

/* src/sort.c */
const SortAlgorithm SORT_ALGORITHMS[] = {
    {"merge", "O(n log n)", "O(n log n)", "O(n log n)", "O(n)", 1, mergeSort},
    {"quick", "O(n log n)", "O(n log n)", "O(n^2)", "O(log n)", 0, quickSort},
    {"heap", "O(n log n)", "O(n log n)", "O(n log n)", "O(1)", 0, heapSort},
};
  
```

층 구조



이때 bench.c와 main.c에는 특정 정렬과 관련한 설명이 한 줄도 없다. SORT_ALGORITHMS 표를 순회할 뿐이다. 그래서 온전히 세 정렬의 결과를 독립적으로 볼 수 있다고 봐도 무방하다고 생각한다. 정렬을 하나 더 넣고 싶다면 그 표에 한 줄을 더하는 것으로 끝난다. tests/test_sort.c 역시 이름을 적지 않고 표를 읽으므로, 새로 넣은 정렬에도 검사가 저절로 붙는다는 점이 편리했다.

파일	말은 것
sort.h	바깥에 공개하는 약속 (Record, SortStats, SortAlgorithm)
sort.c	SORT_ALGORITHMS 표와 결과를 확인하는 공용 함수
mergeSort.c · quickSort.c · heapSort.c	알고리즘 하나씩
bench.c	입력 생성과 시간 측정. 어느 정렬을 재는 중인지는 모른다
main.c	무엇을 썰지 적어 두고 결과를 찍는다 (--csv 를 주면 CSV로)
tools/plot.py	CSV를 읽어 SVG 그래프를 찍는다. 외부 라이브러리를 쓰지 않는다

세 구현이 모두 swap이라는 같은 이름의 함수를 갖고 있는데도 링크가 깨지지 않는다. static을 붙여 파일 밖으로 이름이 새어나가지 않게 했기 때문이다. C에서 파일 단위로 무언가를 감추는 방법이 이것이라는 것을 이번에 알았다.

횃수를 세는 규약

횃수 계산은 카운터를 증가시키는 문장을 알고리즘 중간에 직접 넣어서 책정하였다.

규약	세는 값
대입 1회	이동 1
교환 1회	이동 3
제자리 교환 (i == j)	세지 않는다

처음에는 교환도 이동 1회로 세었는데, 시행착오를 겪으면서 교환 한 번이 tmp = a[i]; a[i] = a[j]; a[j] = tmp 로 대입 세 번이라는 것을 확인하고 3회로 고쳤다. 이 규약을 맞춰야 병합의 이동과 퀵의 이동을 같은 단위로 견줄 수 있다.

그래서 시험 삼아 교본의 예제 배열(2 8 5 9 1 10 7 6 4 3)을 그대로 넣고 돌려 보았고, 비교와 이동 횟수가 교본에 적힌 값과 동일하게 나오는 것을 확인할 수 있었다.

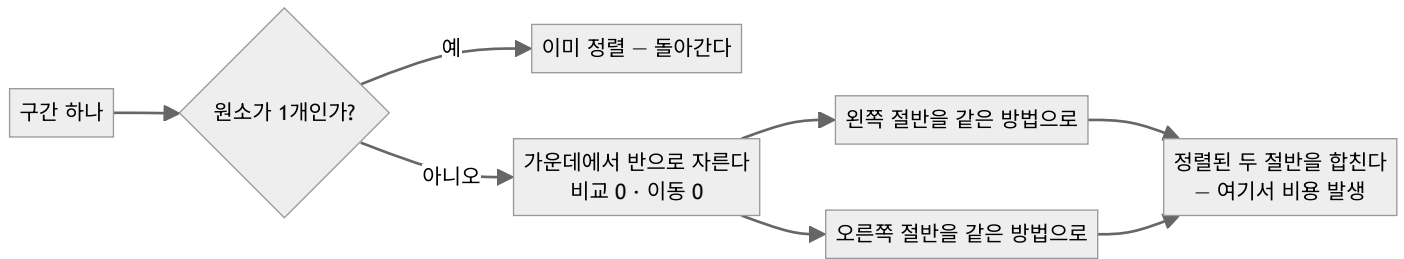
	교본	우리 구현
merge	비교 22 · 이동 68	비교 22 · 이동 68
quick	비교 25 · 이동 21	비교 25 · 이동 21

이와 별도로 make test 가 여러 크기와 입력 모양에서 결과가 옳은지, 원소를 잃지 않았는지, 안정성 주장과 실측이 어긋나지 않는지를 검사한다.

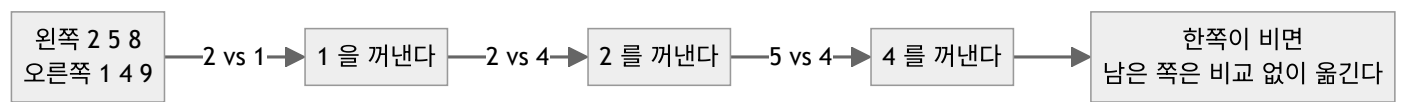
2.2 정렬별 동작 과정

병합 정렬

나누는 것은 사실상 인덱스 산술 한 줄이므로 비용이 들지 않는다고 볼 수 있으나, **합칠 때 비용이 발생하는** 정렬이다. 배열을 원소 하나가 될 때까지 절반씩 나눈 뒤 — 초기 원소 수가 n 이면 $\log_2n$ 번 나뉜다 — 인접한 두 덩어리를 합치며 대소를 비교하여 정렬한다.



합치는 일은 정렬된 두 절반의 맨 앞끼리만 견주어 작은 쪽을 꺼내는 것이다. 그래서 구간 하나를 합치는 데 드는 비교는 구간 크기를 넘지 않는다.



이 과정을 전체 계수 n 에 대하여 $\log n$ 번 반복하므로 시간 복잡도가 $O(n \log n)$ 이다. 그리고 나누는 방식이 값이 아니라 위치이기 때문에 입력이 어떻게 생겼든 반씩 **같**린다. 그래서 최선·평균·최악이 모두 $O(n \log n)$ 으로 같다 — 무너지는 입력이 없다.

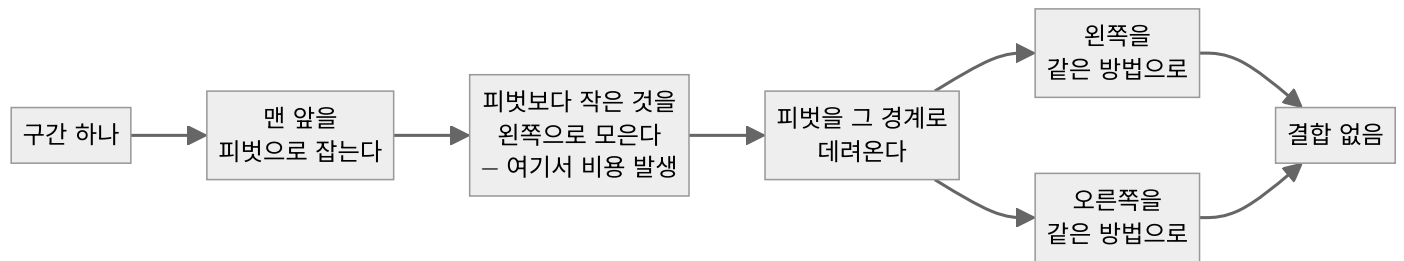
또한 병합 정렬은 **안정성**을 지닌다. 두 값이 같을 경우 $\leq$ 를 통하여 왼쪽 (앞부분)을 먼저 선택하는 방식으로 하여, 원래 배열에서 앞에 있었던 원소를 최종 결과에서도 먼저 배치할 수 있게 되므로 안정적이라고 볼 수 있다.

```

if (a[i].key <= a[j].key) { /* '<' 이면 같은 값일 때 오른쪽이 먼저 나가 불안정해진다 */
    temp[k++] = a[i++];
}
  
```

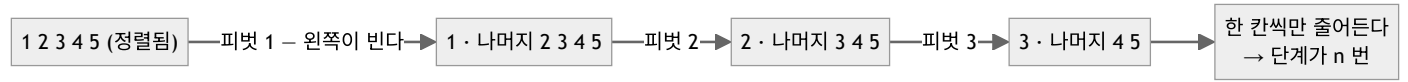
퀵 정렬

합치는 것은 그저 피벗을 기준으로 바로 이어 붙이면 되는 것이기에 비용이 들지 않는다고 볼 수 있으나, **나눌 때는** 피벗을 기준으로 작은 것과 큰 것을 갈라야 하기에 원소 전체 n 을 확인해야 한다. 보통 모든 단계를 수행하면 $\log n$ 이므로 $O(n \log n)$ 이라고 볼 수 있다.



파티션이 피벗의 자리를 확정하면 왼쪽은 모두 피벗보다 작고 오른쪽은 모두 크거나 같다. 두 블록이 최종 배열에서 차지할 구역이 이미 같았으므로 결합은 할 일이 없다. 병합 정렬과 구조가 같은 재귀이지만 **비용이 발생하는 지점이 정반대인** 셈이다.

부가적으로, 퀵 정렬은 **피벗을 어떻게 잡느냐에 따라서 성능이 크게 달라진다**. 이 구현은 강의자료와 같이 구간의 맨 앞 원소를 피벗으로 쓰는데, 그러면 이미 정렬된 입력에서 매번 최솟값이 피벗으로 뽑혀 구간이 한 칸씩밖에 줄지 않는다.



이것이 최악의 경우 $O(n^2)$ 이 되는 이유다. 보완책으로는 피벗을 완전히 랜덤하게 선택하거나, 특정 개수(예: 3개)를 뽑아 그중 중간값을 피벗으로 잡는 median-of-3 방식이 될 수도 있다고 보여진다. **다만 이 과제에서는 그 최악을 실제로 재어 보는 것이 목적 중 하나이므로 일부를 맨 앞 피벗을 그대로 두었다**. 피벗 선택을 바꾸면 최악의 경우가 사라져 관찰할 대상이 없어지기 때문이다.

힙 정렬

배우지 않았던 정렬 방식이다. 어떤 관점에서는 **선택 정렬의 개선안**으로 볼 수 있다. 선택 정렬은 특정 인덱스의 원소를 선택해서 나머지와 모두 비교한 다음, 그 다음 원소를 다룰 때 아까 비교하였던 결과를 사용하는 것이 아니라 다시 버리고 새로 비교를 진행한다. 그러나 힙 정렬은 **비교한 결과를 끝까지 활용하는 정렬**이라고 볼 수 있다.

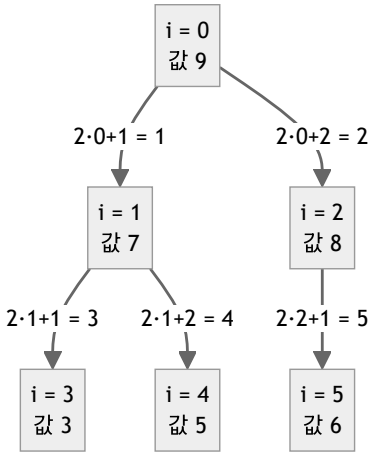
	최댓값 찾기	그 결과를	한 번의 비용
선택 정렬	매번 전체를 확인한다	버린다	$O(n)$
힙 정렬	꼭대기를 본다	힙으로 남겨 둔다	$O(\log n)$ (회복 비용)

힙이라는 구조

힙의 구조는 사실상 트리의 구조로 시각화할 수 있으며, 위에 있는 원소가 부모, 아래에 있는 원소가 자식으로 구성된다. **부모는 항상 자식보다 크거나 같아야 하며, 자식 간의 크기 차이는 없다**. 전체 정렬보다 훨씬 느슨한 조건이라 유지가 싸면서도, 꼭대기가 언제나 최댓값이므로 최댓값을 찾는 비용이 0이다.

여기서 하나 중요하게 짚을 것은, **트리는 따로 존재하지 않는다**는 것이다. 그저 배열을 특정 규칙에 따라 시각화한 것이기에 공간 복잡도가 $O(1)$ 이다. 배열을 위층부터, 왼쪽부터 채웠다고 약속한 가정 아래에서 다음이 성립한다.

관계	인덱스 산술
왼쪽 자식	$2i + 1$
오른쪽 자식	$2i + 2$
부모	$(i - 1) / 2$ (정수 나눗셈)



위 트리가 배열에서는 그냥 [9, 7, 8, 3, 5, 6] 한 줄이다. 포인터가 하나도 없고 인덱스 산술만 있으므로 추가로 잡는 메모리가 없다. 형제인 7과 8 사이에는 아무 순서도 없다는 점도 볼 수 있다 — 힙은 그만큼만 요구한다.

1단계 — 배열을 힙으로 만든다

주어진 배열은 아직 힙이 아니므로 먼저 힙 조건을 만들어 준다. 바닥층(원소의 절반)은 자식이 없어 할 일이 없으므로 그 바로 위 노드부터 거꾸로 올라가며 siftDown 을 부른다. 일이 많은 노드는 개수가 적고 개수가 많은 노드는 일이 적어 서로 상쇄되므로, 이 단계 전체가 $O(n)$ 에 그친다.

2단계 — 꼭대기를 뒤로 보내며 정렬한다

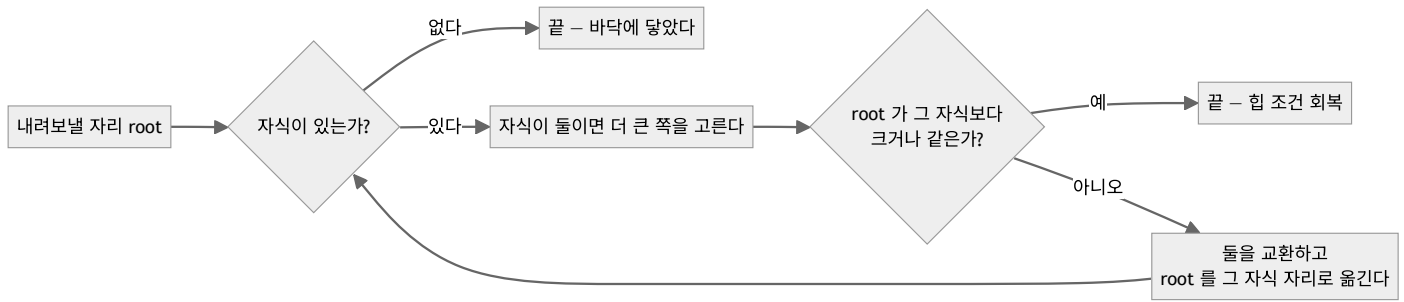
정렬의 과정은 **꼭대기(가장 큰 원소)와 힙 구역의 맨 뒤 원소의 자리를 바꾸면서** 이루어진다. 이 경우 가장 큰 원소는 배열 마지막에 자리를 잡고, 맨 뒤에 있던 원소가 꼭대기로 올라온다.



정렬된 결과가 배열 뒤쪽에 쌓이고 힙은 앞쪽에서 줄어든다. 한 배열을 두 구역으로 나눠 쓰므로 추가 배열이 필요 없다.

siftDown

올라온 원소는 대체로 작아서 힙의 정의(부모는 자식보다 작을 수 없다)를 깨뜨린다. 이를 회복하는 과정이 siftDown 이다. 올라온 수를 아래 자식 수와 비교하여 자식 중 더 큰 것과 자리를 바꾸고, 이것이 성립하지 않을 때까지 지속한다.



자식이 둘일 때 반드시 더 큰 쪽과 교환해야 한다. 작은 쪽을 올리면 그 자리 에서 힙 조건이 곧바로 다시 깨지므로 같은 작업을 반복하게 되기 때문이다.

한 번의 siftDown 은 트리의 높이만큼 내려가므로 log n 단계를 거치며, 이를 n-1 개에 대하여 모두 진행해야 하므로 전체가 o(n log n) 이 나오는 것을 알 수 있다. 그리고 트리의 높이는 입력이 아니라 원소 개수만으로 정해진다. 그래서 힙 정렬은 쿼처럼 무너지는 입력이 없고 최선·평균·최악이 모두 같다.

한편 2단계의 이 교환은 멀리 떨어진 두 원소를 맞바꾸므로 같은 값끼리의 원래 순서를 흐트러뜨린다. 힙 정렬이 불안정한 이유가 여기에 있다.

세 정렬의 시간 복잡도

	최선	평균	최악	무너지는 입력
병합	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	없음 (위치로 나누므로)
퀵	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	정렬됨 · 역순
힙	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	없음 (높이가 n 으로만 정해지므로)

2.3 알고리즘 실험

실험 개요

같은 입력을 세 정렬에 똑같이 먹이고 네 가지를 잰다. 시간은 기계와 부하를 타지만, 비교·이동·재귀 깊이는 입력만 같으면 어디서 돌려도 같은 값이 나온다.

입력은 네 모양으로 만든다. 이론표의 최선·평균·최악 칸을 실험으로 만들어 내는 장치다.

입력	역할
random	평균에 해당
sorted	맨 앞 피벗 퀵의 최악
reversed	또 다른 최악
few-unique (값 8종)	중복 많음 — 안정성을 가릴 수 있는 유일한 입력

n = 8192 로 잡은 이유. 2의 거듭제곱이어야 병합이 바닥까지 정확히 반씩 갈린다. 그렇지 않으면 홀수 구간이 생겨 조각 크기가 들쭉날쭉해지고 실측이 공식과 어긋난다. n = 2¹³에서는 실측이 이론값과 자릿수까지 맞아떨어진다.

항목	공식	값
병합 이동 (모든 입력)	$2n \cdot \log_2 n$	212,992
병합 비교 (정렬된 입력)	$n \cdot \log_2 n / 2$	53,248
퀵 비교 (정렬·역순)	$n(n-1)/2$	33,550,336
퀵 재귀 깊이 (정렬·역순)	n	8,192
힙 재귀 깊이 (전부)	1	1

시간은 평균이 아니라 15회 중 가장 빠른 회차를 쓴다. 측정을 방해하는 것은 시간을 늘리기만 하지 줄이지 못하므로, 최솟값이 이 기계의 실제 성능에 가장 가깝다. 평균을 쓰던 때에는 똑같은 작업을 두 번 켜 값이 3배까지 벌어졌다.

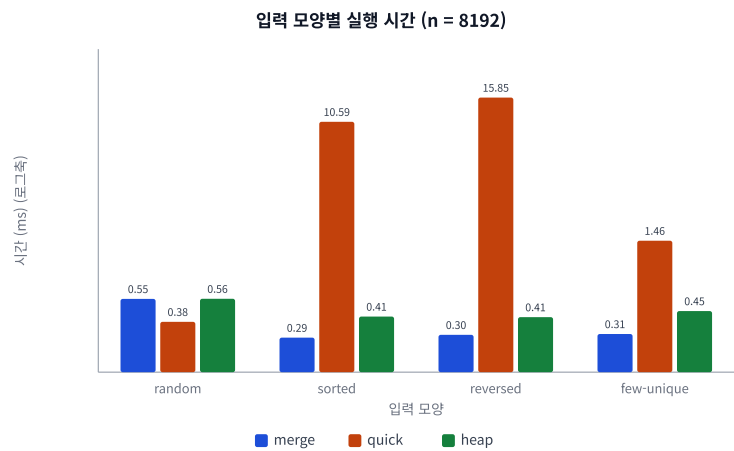
또 퀵의 최악은 나머지 값들의 수백 배라, 세로축을 무엇으로 잡느냐에 따라 같은 자료가 전혀 다르게 보인다. 선형 축은 격차가 얼마나 큰지를 보여 주는 대신 작은 값들은 바닥에 겹쳐 구분되지 않게 하며, 로그 축은 그 반대다. 그래서 비교 횟수만 두 축으로 모두 그렸다.

아래의 표와 그래프는 모두 같은 한 번의 실행에서 나왔고, 원본 수치는 results.csv 에 있다.

입력 모양별 (n = 8,192)

입력	알고리즘	시간(ms)	비교	이동	재귀깊이	안정성
무작위	merge	0.554	96,146	212,992	14	안정
무작위	quick	0.378	129,320	184,500	31	불안정
무작위	heap	0.555	187,630	297,435	1	불안정
정렬됨	merge	0.291	53,248	212,992	14	n/a
정렬됨	quick	10.585	33,550,336	0	8,192	n/a
정렬됨	heap	0.413	194,316	315,918	1	n/a
역순	merge	0.305	53,248	212,992	14	n/a
역순	quick	15.845	33,550,336	12,288	8,192	n/a
역순	heap	0.409	180,760	279,546	1	n/a
중복많음	merge	0.309	92,227	212,992	14	안정
중복많음	quick	1.460	4,222,482	45,957	1,090	불안정
중복많음	heap	0.453	170,282	263,442	1	불안정

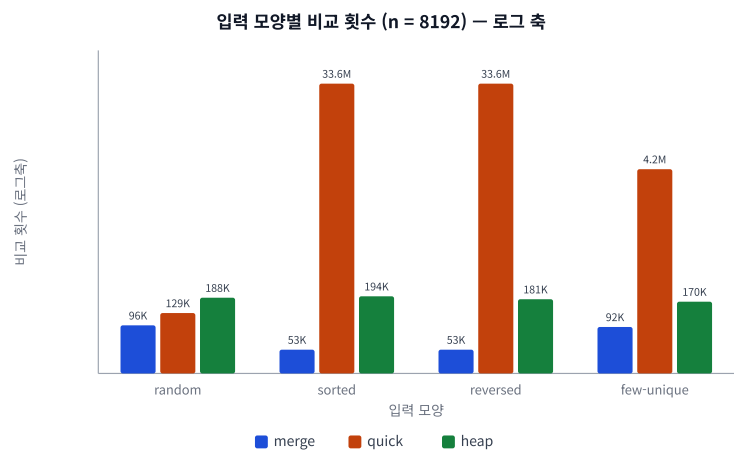
걸린 시간



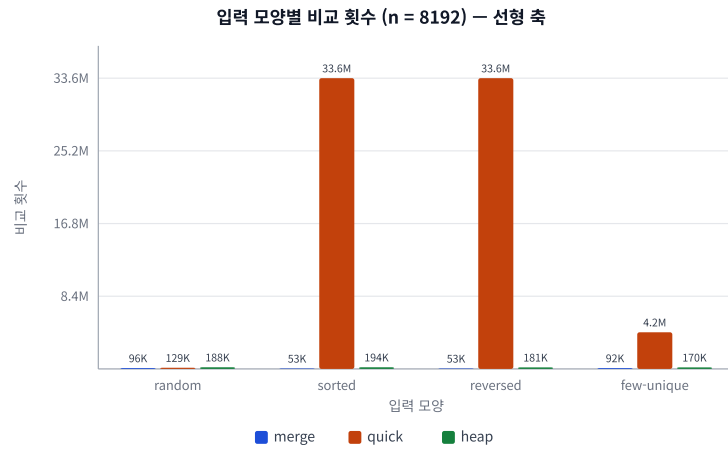
시간 격차는 무작위 입력에서는 거의 나지 않지만, 그 외의 입력에서는 퀵 정렬이 좋지 않은 성능을 보여 주었다. 반대로 병합과 힙 정렬은 어느 입력에서도 예측 가능한 변동 폭 안에서 거의 같은 시간대를 기록하였다.

퀵 정렬이 이런 결과를 보이는 이유의 대부분은, 정렬된 입력이든 역순이든 피벗을 맨 앞 원소로 잡으면 어느 쪽에서도 비효율적으로 동작할 수밖에 없는 구조이기 때문이다. 나눌 때 비용이 드는 정렬이므로, 나누기가 한쪽으로 치우치면 전체 성능이 함께 무너진다.

비교 횟수

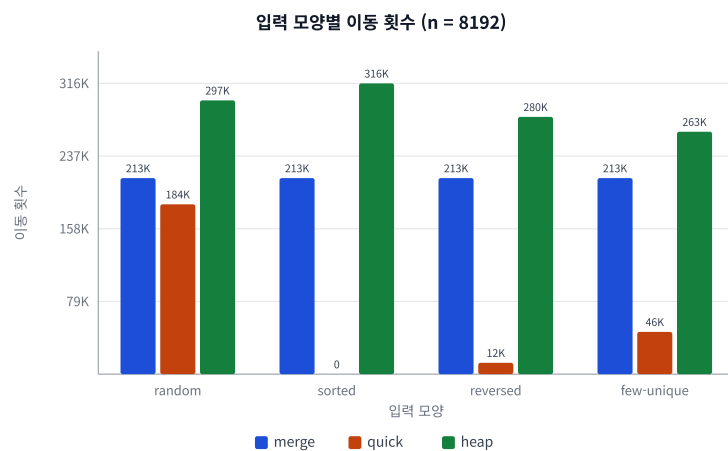


비교 횟수에서 시간 격차의 이유가 더 분명하게 드러난다. 병합·힙과는 비교가 안 될 정도로 퀵의 비교 비용이 크고, 특히 정렬된 입력과 역순에서 그렇다. 앞에서 말한 대로 맨 앞을 피벗으로 잡고 나누는 구조 자체가 매번 남은 원소 전체를 대상으로 진행되기 때문이다. 피벗에 가장 작거나 가장 큰 값이 배정되는 것이 최악이다.



같은 자료를 로그 축이 아닌 선형 축으로 나타낸 것이다. 작은 값들끼리의 차이는 위 그래프보다 덜 보이지만, 퀵 정렬의 막대가 다른 정렬에 비해 월등히 크다는 것은 여전히 그대로 읽힌다.

이동 횟수

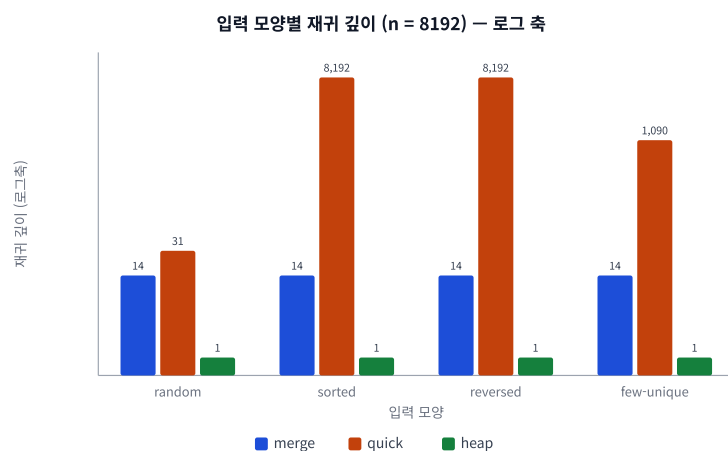


병합은 `temp` 에 빼놓은 값을 다시 같은 개수만큼 원본에 대입해야 하고, 이는 원소가 어떻게 배열되어 있든 변하지 않으므로 항상 동일하다.

퀵은 피벗을 맨 앞에 잡기 때문에 입력 모양을 크게 탄다. 특히 모두 정렬된 경우에는 피벗이 언제나 그 구간의 최솟값이어서 피벗보다 작은 원소가 하나도 없고, 그래서 자리를 바꿀 일 자체가 생기지 않으므로 **0**이다.

ힵ에서 입력 모양에 따른 차이가 크지 않은 이유는, 어찌 되었든 ힵ을 구성해야 하는 것은 동일하고 그것을 구성하고 `siftDown` 하는 과정에서 처음 주어진 배열에 따라 조금씩 차이가 날 뿐이기 때문이다.

재귀 깊이

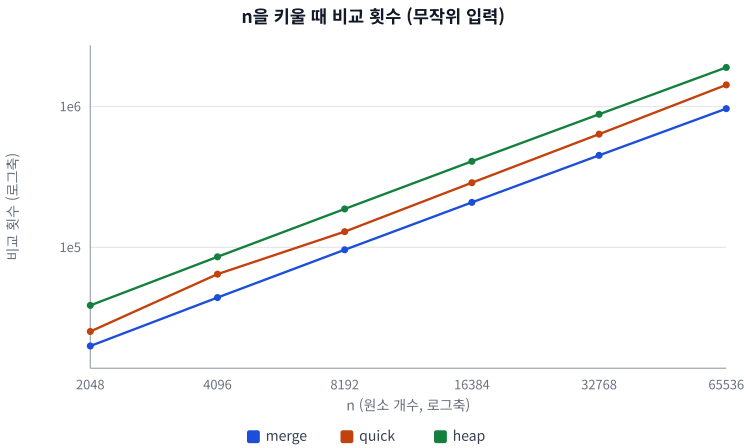
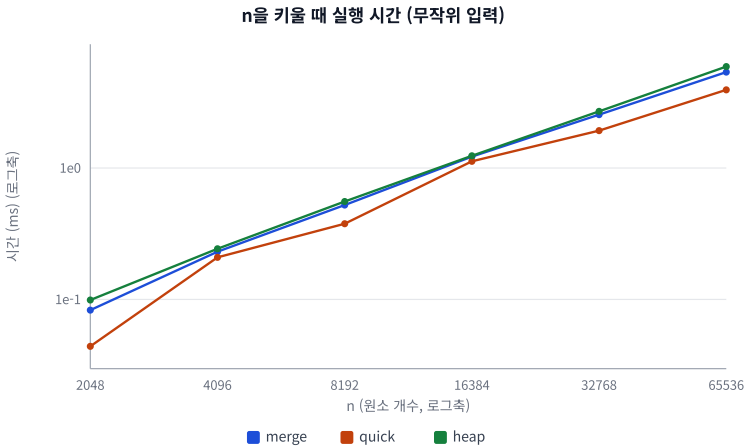


재귀 깊이는 말 그대로 함수가 호출되어 쌓이는 단계의 수로 보면 된다.

- 병합**은 입력과 무관하게 언제나 $\log n$ 만큼 나뉘므로 깊이가 일정하다(14).
- 퀵**은 피벗이 어떤 값으로 잡히느냐에 따라 단계 수가 달라지므로 깊이 또한 달라진다. 무작위에서는 31이지만 정렬·역순에서는 n 인 8,192까지 자란다.
- ힵ**은 1로 표기되어 있는데, 엄밀히 말하면 ힵ 정렬도 $\log n$ 만큼의 단계를 지닌다. 다만 그것을 **반복문으로 한 번에 표현할 수 있으므로** 1이 된다.

n을 키우며 (무작위 입력)

n	merge(ms)	quick(ms)	heap(ms)	merge 비교	quick 비교	heap 비교	merge 깊이	quick 깊이
2,048	0.083	0.044	0.099	19,952	25,272	38,772	12	24
4,096	0.231	0.209	0.243	44,023	64,528	85,695	13	30
8,192	0.523	0.377	0.556	96,146	129,320	187,630	14	31
16,384	1.222	1.126	1.241	208,730	288,062	408,314	15	32
32,768	2.552	1.927	2.700	450,210	636,910	882,006	16	38
65,536	5.374	3.935	5.917	965,821	1,424,177	1,894,624	17	46



마지막으로, n을 키움에 따라 실행 시간과 비교 횟수 모두 예상대로 크게 증가하는 것을 볼 수 있었다.

안정성 판정

안정성은 한 번 돌려서는 결론 낼 수 없다. 두 가지 함정이 있다.

- 1. 중복 key가 없으면 판정 자체가 불가능하다. 정렬됨 · 역순 에서는 어떤 정렬이든 결과가 같아지므로, 그때의 "안정함"은 판정이 아니라 우연이다. 그래서 위 표에서 n/a로 적었다. 안정 으로 적으면 "확인됐다"로 오해된다.
- 2. 한 번의 시도에서 불안정 정렬이 우연히 원래 순서를 지킬 수 있다.

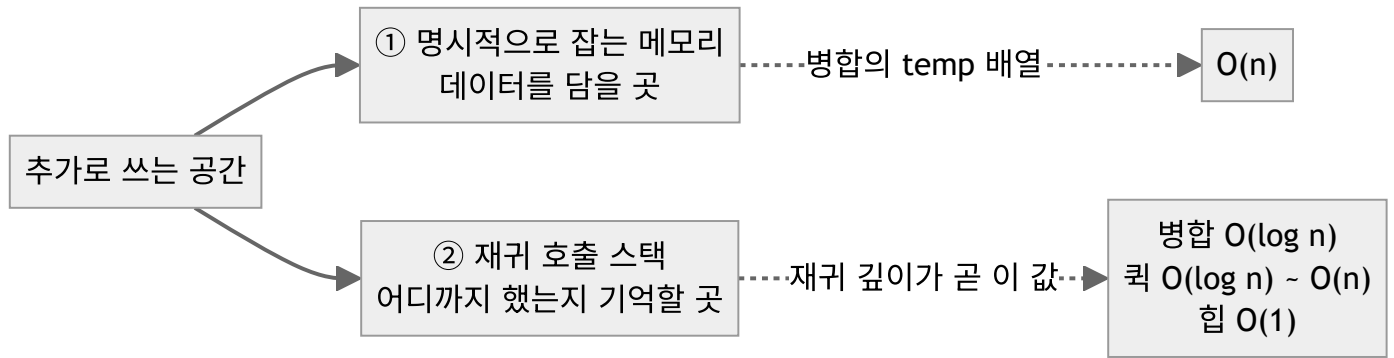
그래서 중복이 많은 입력(값 8종, n = 1024)을 시드 200개로 돌려 한 번이라도 순서가 깨지면 불안정으로 판정하였다.

알고리즘	표의 주장	안정으로 나온 시도	판정
merge	안정	200 / 200	안정
quick	불안정	0 / 200	불안정
heap	불안정	0 / 200	불안정

2.4 공간 복잡도

추가적으로 생각해 볼 만한 것은 공간 복잡도다. 기본적으로 배운 것들이 모두 시간 복잡도와 관련된 것들이라 공간 복잡도에 관한 궁금증이 생겨서 조금 더 찾아보았다.

공간 복잡도란 알고리즘이 문제를 해결하기 위해 실행되는 동안 추가로 사용하는 메모리의 최대량으로 볼 수 있으며, 기본적으로 명시적으로 요구되는 추가 메모리와 재귀 호출 스택 두 가지 요소가 주요하게 작용한다.



이 관점에서 정렬 알고리즘들을 조금 더 자세히 뜯어보면 다음과 같다.

- **병합 정렬** — 명시적으로 원소 개수에 따른 배열을 따로 저장해야 하기에 n 의 공간이 필요하며, 추가적으로 $\log n$ 만큼 단계가 있기에 재귀 스택을 고려하면 최종적으로 $O(n)$ 으로 볼 수 있다. 두 항 중 큰 쪽이 n 이므로 $O(n) + O(\log n) = O(n)$ 이다.
- **퀵 정렬** — 명시적으로 다른 메모리가 필요한 것은 아니며, 재귀 스택의 경우만 피벗이 어느 지점에 잡히느냐에 따라서 $\log n \sim n$ 으로 변동이 있다고 볼 수 있다.
- **힙 정렬** — 트리 구조가 실제로 존재하는 것이 아니라 주어진 배열이 특정 규칙에 따라 정렬된 것이므로 따로 명시적인 메모리가 요구되지 않으며, 단계가 $\log n$ 만큼 존재하기에 $\log n$ 으로 볼 수도 있으나, 재귀 깊이 부분에서도 보았듯이 **반복문으로 구현한다면 $O(1)$ 로도 가능**하므로 셋 중에 힙 정렬이 공간 복잡도는 가장 낮다고 볼 수 있다.

알고리즘	① 명시적 메모리	② 재귀 스택	합계
merge	$O(n)$ (temp 배열)	$O(\log n)$	$O(n)$
quick	없음	$O(\log n) \sim O(n)$	$O(\log n) \sim O(n)$
heap	없음	$O(1)$ (반복문)	$O(1)$

이 부분을 좀 더 공부해 보면서, 데이터가 커짐에 따라서 단순히 연산 시간뿐 아니라 쓰이는 메모리까지 어떻게 변하는지 검토해 볼 수 있었다. 또한 전에는 재귀 함수의 반복이 많아졌을 때 느려지거나 꺼지는 것을 **연산량이 많아져서** 그런 줄 알았는데, **재귀 스택 때문일 수도 있겠다**는 시각까지 얻을 수 있게 되었다.

3. 소감 및 결론

결론적으로 봤을 때, **세 가지 알고리즘은 각각 한 가지씩은 포기했다**고 볼 수 있다. 세 정렬 모두 평균은 $O(n \log n)$ 이다. 그렇지만,

- **병합**의 경우는 해당 복잡도를 달성하기 위하여 temp 배열 n 개를 따로 저장해야 하는 메모리 소모가 다른 정렬들에 비해 필요하였다.
- **퀵**은 메모리를 거의 안 쓰고 통상적으로 가장 빠른 정렬 중 하나이지만, 최악의 경우 $O(n^2)$ 이 존재한다. 전부 정렬된 케이스 같은 특정 상황에서는 예상된 결과 도출 시간을 초과하는 문제가 발생한다. 실제로 $n = 8,192$ 에서 정렬된 입력을 주자 무작위 입력의 **28배**(0.378ms → 10.585ms)가 걸렸다.
- **힙**의 경우는 최악도 $O(n \log n)$ 이며 메모리도 $O(1)$ 이라 모두 괜찮아 보이지만, **무작위 입력에서는 실제 측정 속도가 가장 느린 것으로 드러났다**. 비교 횟수가 병합의 약 2배(187,630 대 96,146)인 것이 그 원인으로 보인다.

이에 따라서, 힙 정렬은 메모리와 최악 보장은 있으나 평균적인 실측 속도가 부족하기에 **세 가지 모두 한 가지씩의 결점은 지니고 있음**을 알 수 있다.

	포기한 것	얻은 것
병합	추가 메모리 $O(n)$ · (안정성은 지킴)	입력을 타지 않는 $O(n \log n)$ · 안정
퀵	최악 $O(n^2)$ · 불안정	평균 최고 속도 · 메모리 거의 0
힙	평균 속도 (상수가 크다)	최악도 $O(n \log n)$ · 메모리 $O(1)$

이 프로젝트를 하면서 평소 딱히 생각지도 않았던 정렬 알고리즘을 상대적으로 깊게 탐색하며 여러 경우를 나눠서 분석해볼 수 있었다는 것이 좋았다. 특히, 수업에서 배우지 않았던 정렬과 배운 정렬을 서로 연계 및 비교하면서 훨씬 더 시간 복잡도와 알고리즘 구조에 관하여서 훨씬 더 친숙해질 수 있었다고 생각한다.

개인적으로, 교수님께서 강조하셨던 바이브 코딩을 적극 활용하여 보았는데, 코딩 부분은 AI와 함께하되 **알고리즘의 구조와 그와 관련된 부분은 확실히 스스로 AI와 함께 학습할 수 있었기에** 선택과 집중을 통한 효율적 학습을 성취한 것 같아 흥미로웠다.